

OPERATING SYSTEMS

Pintos: Report Project #6

Group #6

Davide Frova, Costanza Rodriguez Gavazzi

Fall Semester, May 20, 2024

1 FILES CHANGED

- `/pintos/threads/thread.h` (*modified*)
- `/pintos/threads/thread.c` (*modified*)
- `/pintos/userprog/process.c` (*modified*)
- `/pintos/userprog/syscall.c` (*modified*)

2 CHANGES

`/pintos/threads/thread.h`

Integrated given solution

`/pintos/devices/thread.c`

Integrated given solution

`/pintos/userprog/process.c`

Integrated given solution

/pintos/userprog/syscall.c

- *(added)* **imports**
- *(added)* method **signatures** for newly implemented methods
- *(added)* **struct item {int fd; struct file *file; struct hash_elem elem;}**: Needed to create hash table to map ints (fds) to file pointers. We decided to go with the global table shared between threads (with a lock to access it)
- *(added)* **method unsigned item_hash(const struct hash_elem *e, void *aux)**: required for hashtable definition and usage
- *(added)* **method bool item_compare(const struct hash_elem *a, const struct hash_elem *b, void *aux)**: required for hashtable definition and usage
- *(added)* **global variable struct hash file_table**: the actual hash table
- *(added)* **global variable struct semaphore file_table_lock**: the lock to protect the table
- *(added)* **global variable unsigned next_fd**: a number to keep track of the unique hash table entries (files)
- *(modified)* **void syscall_init (void)**:
 - Added initialization of the hashtable and the lock
 - Added the syscall method references in the call[]
- *(added)* **method static void syscall_halt (struct intr_frame *f)**: Calling shutdown_power_of()
- *(added)* **method static void syscall_create (struct intr_frame *f)**:
 - Acquire the lock to access the file table
 - Get the file name from the stack
 - Check if the pointer to the filename is valid using provided method
 - Get the initial size of the file from the stack
 - Create the file and store the result of this operation in a local variable
 - Put the result of the creation of the file in f->eax
 - Release the lock
- *(added)* **method static void syscall_remove (struct intr_frame *f)**:
 - Acquire the lock to access the file table
 - Get the file name from the stack
 - Check if the pointer to the filename is valid using provided method
 - Remove the file and store the result of this operation in a local variable
 - Put the result of the operation in f->eax
 - Release the lock
- *(added)* **method static void syscall_filesize (struct intr_frame *f)**:
 - Acquire the lock to access the file table
 - Get the file descriptor (fd) from the stack

Get the corresponding file from the hashtable

Check if what was retrieved is null, in that case release the lock and return the error code on f->eax

Otherwise get the file pointer from the hash table entry that we got

Get the filesize using the provided method file_length

Put the result of the operation in f->eax

Release the lock

- *(added) method static void syscall_close (struct intr_frame *f):*

Acquire the lock to access the file table

Get the file descriptor (fd) from the stack

Get the corresponding file from the hashtable

Check if what was retrieved is null, in that case release the lock

Otherwise get the file pointer from the hash table entry that we got

Close the file using the provided method file_close

Remove the entry from the hash table

Release the lock

- *(added) method static void syscall_tell (struct intr_frame *f):*

Acquire the lock to access the file table

Get the file descriptor (fd) from the stack

Get the corresponding file from the hashtable

Check if what was retrieved is null, in that case release the lock and return the error code on f->eax

Otherwise get the file pointer from the hash table entry that we got

Get the file position using the provided method file_tell

Put the result of the operation in f->eax

Release the lock

- *(added) method static void syscall_seek (struct intr_frame *f):*

Acquire the lock to access the file table

Get the file descriptor (fd) from the stack

Get the file position from the stack

Get the corresponding file from the hashtable

Check if what was retrieved is null, in that case release the lock

Otherwise get the file pointer from the hash table entry that we got

Seek the file using the provided method file_seek

Release the lock

- *(added) method static void syscall_read (struct intr_frame *f):*

Acquire the lock to access the file table

Get the file descriptor (fd) from the stack

Get the buffer pointer from the stack

Get the buffer size from the stack

Check the validity of the buffer pointer, if it's invalid release the lock and return the error code on f->eax

Check if the file descriptor is 0 (stdin), if yes then we are reading from the keyboard so we fill the buffer using input_getc(), then we push the size on f->eax and we release the lock.

Otherwise, get the corresponding file from the hashtable

Check if what was retrieved is null, in that case release the lock and return the error code on f->eax

Otherwise get the file pointer from the hash table entry that we got

Read from the file and save the result in the buffer using the provided method file_read

Put how many bytes we read in f->eax

Release the lock

- (added) **method static void syscall_open (struct intr_frame *f):**

Acquire the lock to access the file table

Get the file pointer from the stack

Check the validity of the file pointer, if it's invalid release the lock and return the error code on f->eax

Otherwise, open the file with provided method filesys_open and save its pointer in file_opened

Check if the file was opened, if not then release the lock and return the error code on f->eax

Otherwise, create a new struct item *file_item in order to put the file in the hash table

Set the fd as the next_ft and the file pointer to the file_opened

Insert the newly created element in the hashtable

Put the next_fd in f->eax (because we return the fd of the newly opened file)

Increment the global next_fd

Release the lock